Audion Docs AI - User Guide

Audion Docs AI audits, normalizes, and transforms DOCX/PPTX document sets through reproducible local pipelines with optional AI-provider stages.

Start

Launch the GUI for normal work. Keep source documents in a dedicated input folder and choose a separate output. Verify the portable runtime before processing a large set.

The GUI is an operator shell over project services. It selects files, parameters, templates, and providers; the document logic remains in `system_core\` and must behave the same from CLI.

Recursive Document Pipeline

1. Add DOCX and/or PPTX sources.
2. Select the target folder.
3. Run `Scan` to inventory documents and supported objects.
4. Build the render map when visual or structural anchors are required.
5. Run the selected audit.
6. Review the report before annotation or normalization.
7. Apply approved changes to copies.
8. Strip temporary anchors only after verification.

Do not skip directly from source selection to destructive normalization. The scan and audit reports are the evidence for every applied correction.

Audit Results

The audit may report formatting anomalies, inconsistent styles, suspicious punctuation or case, broken document structure, unsupported objects, and provider-specific warnings. Separate definite defects from suggestions.

Keep the original text and object location in reports. Automated cleanup must not silently rewrite uncertain content.

Document TASK Workflow

Document TASK operations build structured deliverables from templates, exact replacements, or extracted facts.

- Select the source set and the approved template.
- Confirm output format and required fields.
- Use exact replacements for deterministic edits.
- Use AI extraction only where the task explicitly permits interpretation.
- Validate tables, headings, page structure, and missing values in the result.

A template defines the output contract. Do not improvise layout or rename required fields without changing the task specification.

Normalization

Normalization is applied only after audit review. Typical operations include style cleanup, spacing, punctuation, controlled case restoration, removal of service anchors, and correction of known structural defects.

Run on copies. Compare the result visually and structurally, and preserve the report that explains each class of change.

Providers And Keys

Provider keys live in project `config\api_key_*.txt` files and are never printed to logs or embedded in public documentation. Select the provider and model appropriate for the task. Provider output is input to validation, not automatic truth.

For public archives, keys are replaced only in isolated staging by the release tooling. Working keys remain unchanged.

Workbench And Output

`Source` contains the managed document list; `Target` is the output folder. `Reset` clears the current selection and `Delete` removes only the selected list entry. Paths with spaces and Cyrillic are supported.

Reports, annotated copies, normalized copies, extracted tables, and service maps are written to their managed output/report locations. Do not use the project root as a work folder.

Verification

Before accepting a run:

- compare input and output document counts;
- open representative DOCX/PPTX files;
- inspect tables, images, notes, headers, and page breaks;
- review skipped and unsupported objects;
- confirm temporary anchors were removed when required;
- confirm provider errors did not become accepted content;
- retain logs and the audit report.

Troubleshooting

- Empty scan: verify source selection and supported extensions.
- Missing objects: inspect the unsupported-object section of the report.
- Bad Cyrillic or launcher loop: run the CMD encoding fixer and compare FZF/fallback menus.
- Provider failure: verify key, model, network availability, and retry policy.
- Output differs visually: review render-map anchors and normalization scope.
- Locked temporary files: close the previous process and remove only managed temp artifacts.

Safe Maintenance

Cleanup may remove caches, temporary render maps, logs, reports, and rebuildable runtime according to policy. It must preserve source code, config, input, output, templates, keys, and canonical documentation.

Keep the approved template, provider/model choice, normalization scope, and audit report with every accepted batch so the transformation can be reproduced.